

VIT BHOPAL UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT

PYTHON-BASED TO-DO LIST MANAGEMENT SYSTEM

A terminal-based application for organizing academic tasks

Submitted by

Sarthak Sanekar

B.Tech – Computer Science and Engineering

Specialisation: Artificial Intelligence and Machine Learning

University: VIT Bhopal University

Academic Year: 2026–2030

Faculty / Course Instructor: **Dr. D Lakshmi**

Date of Submission: **30 Sept. 2026**

Abstract

This project is a menu-driven To-Do List Management System developed in Python. It allows a user to add tasks, view all tasks, mark tasks as completed, edit task names, delete tasks after confirmation, and display only pending tasks. The application runs in a terminal and stores tasks in memory using a list of dictionaries. It demonstrates functions, loops, conditional statements, lists, dictionaries, input validation, and modular design.

1. Introduction

Students manage assignments, presentations, coding work, and other academic responsibilities. A To-Do List provides a simple way to record activities and review progress. This project implements essential task-management operations in a lightweight command-line application without external libraries or a database.

2. Problem Statement

Informal task tracking can make it difficult to review unfinished work or identify completed activities. The goal is to create a small program that keeps a structured task list and provides basic operations through a menu.

3. Objectives

- Provide a clear menu for navigating the application.
- Add tasks and reject blank task names.
- Display numbered tasks with Pending/Done status.
- Mark, edit, or delete selected tasks.
- Ask for confirmation before deletion.
- Show pending tasks separately.
- Practice modular Python programming and input validation.

4. Scope and Limitations

The application is designed for one user in a terminal session. Task data is stored in memory and cleared when the program closes. The current version does not include accounts, due dates, reminders, a graphical interface, or cloud synchronisation.

5. Tools and Technologies

- Programming language: Python 3
- Interface: text-based terminal / command prompt
- Data structure: list containing dictionaries
- Design: functions and a menu-controlled main loop
- External packages: none required for the application

6. Main Features

View all tasks: Displays every task with its number, status, and totals.

Add a task: Accepts a non-empty task name and adds it with Pending status.

Complete a task: Changes the selected task status to Done.

Edit a task: Allows the user to change an existing task name.

Delete a task: Removes a selected task after confirmation.

View pending tasks: Displays only tasks that are not yet completed.

Exit: Ends the menu loop and closes the program.

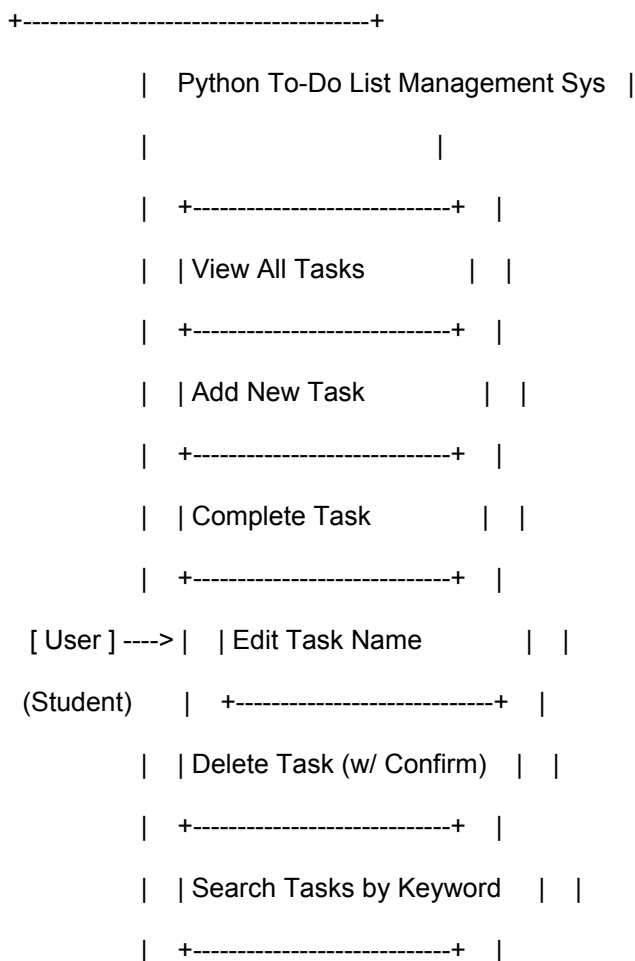
7. Program Design and Logic

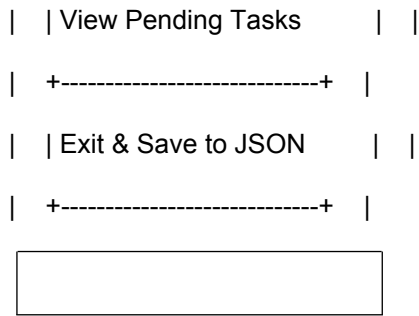
The program starts with an empty tasks list and displays a menu. The main loop accepts a choice and calls the corresponding function. Operations that select an existing task use a shared helper to display the list, validate integer input, check the allowed range, and return a zero-based index or None.

Each task is represented by a dictionary containing a name (text) and done (Boolean). A list stores the dictionaries in the order tasks were added. This makes it straightforward to update a task's name or completion status.

7.1. Use Case Diagram

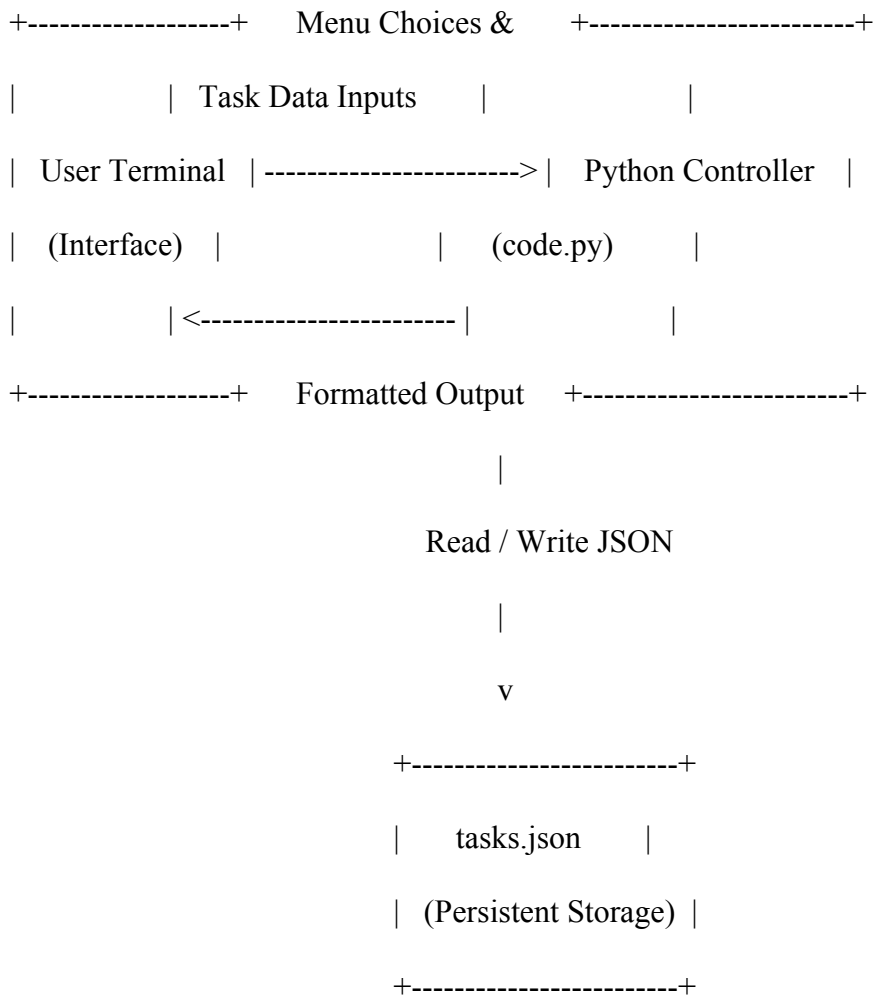
This diagram illustrates the interaction between the user (student) and the application boundary, incorporating all core features (including search and persistent management).





7.2. System Architecture & Data Flow Diagram (DFD Level 0)

This diagram shows how user inputs flow through the Python functions and interact with local persistent storage.



7.3. Activity Diagram / Program Flowchart Logic

This maps out the sequential execution logic of the application loop from startup to termination.

[Start Program]

|



[Load Tasks from tasks.json]



[Display Menu]



[Get User Choice (1-8)]



└─▶ Case 1: View All Tasks

└─▶ Case 2: Add Task

└─▶ Case 3: Complete Task

└─▶ Case 4: Edit Task

└─▶ Case 5: Delete Task

└─▶ Case 6: Search Tasks

└─▶ Case 7: View Pending Tasks



[Execute Selected Function]



[Save Updated State to tasks.json]



└─▶ Choice != 8



▼ (Choice == 8)

[Exit Program / Goodbye]

|

▼

[End Program]

7.4. Terminal UI Wireframe Layout

A visual mockup of how the command-line interface renders for the user:

```
=====
                        TO-DO LIST
=====

1. View all tasks
2. Add a task
3. Complete a task
4. Edit a task
5. Delete a task
6. Search tasks
7. View pending tasks
8. Exit

=====

Choose an option (1-8):
```

8. Implementation Overview

- `show_tasks()` displays all tasks and status totals.
- `add_task()` accepts a non-empty name and creates a Pending task.
- `get_task_number()` validates selection and returns an index or None.
- `complete_task()` marks a selected task as Done.
- `edit_task()` updates a selected task's name.
- `delete_task()` asks for confirmation before removing a task.

- `show_pending()` filters and displays unfinished tasks.
- `show_menu()` prints choices; `main()` controls the loop.

The entry-point check starts `main()` when the Python file is run directly. Separating operations into functions improves readability and makes the program easier to test.

9. Demonstration Screenshots and Outcomes

These screenshots document a sample run and demonstrate the add, view, complete, and delete operations.

Screenshot 1 — Add a task

Option 2 is selected. The user enters “Maths Homework” and the program confirms that the task was added successfully.

```
Welcome to your To-Do List!

=====
                TO-DO LIST
=====

1. View all tasks
2. Add a task
3. Complete a task
4. Edit a task
5. Delete a task
6. View pending tasks
7. Exit
=====

Choose an option (1-7): 2

--- ADD TASK ---
Enter task name: Maths Homework
Task added successfully!
```

Screenshot 2 — View all tasks

Option 1 displays the current tasks: Maths Homework, CSE Project, and EVS PPT making. All three are shown as Pending.

```
=====
                        TO-DO LIST
=====
1. View all tasks
2. Add a task
3. Complete a task
4. Edit a task
5. Delete a task
6. View pending tasks
7. Exit
=====
Choose an option (1-7): 1

===== MY TO-DO LIST =====
1. [Pending] Maths Homework
2. [Pending] CSE Project
3. [Pending] EVS PPT making
```

Screenshot 3 — Complete a task

Option 3 is selected and task number 3 is entered. The program confirms completion; EVS PPT making is then displayed as Done.

```
=====
                        TO-DO LIST
=====
1. View all tasks
2. Add a task
3. Complete a task
4. Edit a task
5. Delete a task
6. View pending tasks
7. Exit
=====
Choose an option (1-7): 3

--- COMPLETE TASK ---

===== MY TO-DO LIST =====
1. [Pending] Maths Homework
2. [Pending] CSE Project
3. [Pending] EVS PPT making
Enter task number: 3
Task marked as completed!
```

Screenshot 4 — Delete a task

Option 5 is selected and task number 3 is chosen. After the user confirms with “y”, the program reports that EVS PPT making was deleted.

```

===== MY TO-DO LIST =====
1. [Pending] Maths Homework
2. [Pending] CSE Project
3. [Done] EVS PPT making

=====

                TO-DO LIST
=====
1. View all tasks
2. Add a task
3. Complete a task
4. Edit a task
5. Delete a task
6. View pending tasks
7. Exit
=====
Choose an option (1-7): 5

--- DELETE TASK ---

===== MY TO-DO LIST =====
1. [Pending] Maths Homework
2. [Pending] CSE Project
3. [Done] EVS PPT making
Enter task number: 3
Delete this task? (y/n): y
Deleted task: EVS PPT making

```

10. Testing and Validation

The screenshots show successful operation of the main workflow. Additional testing should cover invalid menu options, blank task names, non-numeric or out-of-range task numbers, declining deletion, and operations when the list is empty.

Test case	Expected behaviour	Evidence / status
Add a valid task	Added as Pending	Screenshot 1
View task list	Shows tasks and statuses	Screenshot 2
Complete task #3	Selected task becomes Done	Screenshot 3
Confirm deletion of task #3	Selected task is removed	Screenshot 4
Blank name / invalid number	Input handled safely	Additional test recommended

11. Results and Discussion

The demonstration shows the central task lifecycle: creating a task, reviewing the list, changing a task's status, and removing a task with confirmation. Numbering supports selection, while status labels make progress visible.

12. Future Enhancements

- Save and load tasks using JSON so they persist between sessions.
- Add due dates, priorities, and categories.
- Add search and sorting options.
- Create a graphical interface using Tkinter or a web interface.
- Add automated tests for validation and task operations.

13. Conclusion

The Python To-Do List Management System provides a simple terminal interface for managing academic tasks. It demonstrates how core Python data structures and control flow can solve a practical problem. File-based storage would be a useful next improvement.

References

Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>

Project source code: Student-developed Python terminal application.

Vityarthi : Python Essentials